Jsp - Servlet

Chapitre 2: Récupération d'informations

Dr. Ahmad Khoureich Ka

UADB

L3 D2AW

1. Introduction
2. Les paramètres d'initialisation
3. Les informations contenues dans la requête
   - Les données de formulaires
   - Informations sur la machine cliente
   - Informations sur l'utilisateur
   - Les en-têtes HTTP de requête
4. Les informations sur le serveur
5. Autres informations
   - Attributs de requête
   - Attributs de contexte

Introduction

- Pour bien répondre à la requête du client une application web doit pouvoir accéder à certaines informations, parmi lesquelles il y'a :

1. Les paramètres d'initialisation.
2. Les informations contenues dans la requête :
   - les données de formulaires,
   - les spécificités du client
   - les en-têtes HTTP de requête.
3. Les informations sur l'environnement dans lequel elle est exécutée :
   - caractéristiques de la machine serveur,
   - caractéristiques du logiciel serveur.

Les paramètres d'initialisation

- Ce sont des paramètres définis dans le descripteur de déploiement (**`web.xml`**) ou à l'aide d'annotations au niveau de la classe du servlet.

- Un paramètre d'initialisation de servlet n'est accessible que pour le servlet pour lequel il est défini. Il est récupérable à partir d'un objet **`ServletConfig`**.
  Interface **`javax.servlet.ServletConfig`**

- Un paramètre d'initialisation de contexte est accessible pour tous les servlets de l'application web. Il est est récupérable à partir d'un objet **`ServletContext`**.
  Interface **`javax.servlet.ServletContext`**

Les paramètres d'initialisation

Exemple :

☞ Définition d'un paramètre d'initialisation de servlet dans **web.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app ...>
   ...
   <servlet>
      <servlet-name>HtmlServlet</servlet-name>
      <servlet-class>htmlservlet.HtmlServlet</servlet-class>
      <init-param>
         <param-name>nbLignes</param-name>
         <param-value>5</param-value>
      </init-param>
   </servlet>
   ...
</web-app>
```

Les paramètres d'initialisation

- Définition d'un paramètre d'initialisation de servlet à l'aide d'annotations

```
@WebServlet(
        name = "HtmlServlet",
        urlPatterns = {"/HtmlServlet", "/HServlet"},

        initParams = {
           @WebInitParam( name = "nbLines", value = "5" ),
           @WebInitParam( name = "nbRows", value = "45" )
        }
)
public class HtmlServlet extends HttpServlet {
    ...
}
```

- Récupération du paramètre d'initialisation
  Méthode de la classe `javax.servlet.GenericServlet`

```
String nbRows = getServletConfig().getInitParameter("nbRows");
```

Les paramètres d'initialisation

- Définition d'un paramètre d'initialisation de contexte

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app ...>
   <context-param>
      <param-name>dbUrl</param-name>
      <param-value>jdbc:mysql://localhost:3306/scol</param-value>
   </context-param>
   ...
</web-app>
```

- Récupération du paramètre de context
  Méthode de la classe **`javax.servlet.GenericServlet`**

```
String dbUrl = getServletContext().getInitParameter("dbUrl");
```

Les paramètres d'initialisation

Remarque

- Les objets **`ServletConfig`** et **`ServletContext`** disposent de la méthode :
  `Enumeration<String> getInitParameterNames()`
  qui retourne les noms des paramètres d'initialisation (associés respectivement au servlet et au contexte).

Exemple

- Affichage de la liste des paramètres d'initialisation du contexte

```
Enumeration<String> listParam =
    getServletContext().getInitParameterNames();
while(listParam.hasMoreElements()){
    out.println(listParam.nextElement());
}
```

Les informations contenues dans la requête

Ces informations regroupent :

- les données de formulaires,
- les informations sur la machine cliente,
- les informations sur l'utilisateur,
- les en-têtes HTTP de requête

Les données de formulaires

- Les formulaires permettent à l'utilisateur d'échanger des informations avec l'application.

- Les données de formulaires sont intégrées à la requête (sous forme de paramètres) soit :
  - avec la méthode `get` où, elles sont ajoutées à la fin de l'url après le point d'interrogation.
  - avec la méthode `post` où elles sont séparées de l'url.

Les données de formulaires

Requête **GET** et requête **POST**

- requête **GET**

```
GET /nom_du_servlet?param1=valeur1&param2=valeur2&... HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/...
Accept: text/html,...
Referer: ...
  ...
```

- requête **POST**

```
POST /nom_du_servlet HTTP/1.1
Host: localhost:8080\r\n
User-Agent: Mozilla/...
Accept: text/html,...
  ...
param1=valeur1&param2=valeur2&...
```

Les données de formulaires

- Les paramètres de requête peuvent ne pas avoir de valeur ou avoir une ou plusieurs valeurs.

- Quoiqu'ils puissent être intégrés différemment à la requête, les paramètres de requête sont récupérés avec les mêmes méthodes de l'interface `javax.servlet.http.HttpServletRequest` :
  - `getParameter(String nom)`
  - `getParameterValues(String nom)`
  - `getParameterNames()`

getParameter(String nom)

- La méthode

```
public String getParameter(String name)
```

est utilisée pour les paramètres monovalués. Elle prend en argument le nom du paramètre dont elle doit retourner la valeur.

- La valeur retournée est :
  - une chaine de caractères (`String`) si le paramètre a une valeur,
  - une chaine de caractères vide si le paramètre n'a pas de valeur,
  - `null` si le paramètre n'existe pas dans la requête.

Remarque :

Il est conseillé de tester l'existence de la valeur retournée par `getParameter` avant de l'utiliser.

getParameter(String nom)

Exemple :

Dans un formulaire, une zone de saisi :

```
Nom: <input type='text' name='nom'/>
```

Dans un servlet, la récupération de la valeur saisie :

```
String nom = request.getParameter("nom");
```

getParameterValues(String nom)

- La méthode

```
public String[] getParameterValues(String name)
```

est utilisée pour les paramètres multi-valués. Elle prend en argument le nom du paramètre dont elle doit retourner les valeurs.

- Cette méthode retourne :
  - toutes les valeurs du paramètre dans un tableau de chaine de caractères (`String[]`),
  - un tableau ne contenant que des chaines de caractères vides si le paramètre n'a aucune valeur,
  - `null` si le paramètre n'existe pas dans la requête.

Remarque :

Il est conseillé de tester l'existence de la valeur retournée par `getParameterValues` avant de l'utiliser.

getParameterValues(String nom)

Exemple :

```
Donnez 3 fruits<br/>
<input type='text' name='fruits' /> <br/>
<input type='text' name='fruits' /> <br/>
<input type='text' name='fruits' /> <br/>
```

Dans un servlet, la récupération des valeurs :

```
String[] fruits = request.getParameterValues("fruits");
```

getParameterNames()

- La méthode **`getParameterNames()`** permet d'obtenir la liste des paramètres contenus dans la requête. Elle retourne un objet de type **`java.util.Enumeration`**.

- Exemple :
  Quelques champs d'un formulaire :

```
Nom: <input type='text' name='nom'/><br/>
Prénom: <input type='text' name='prenom'/><br/>
Civilité:
<input type='radio' name='civil' value='h'/>M.
<input type='radio' name='civil' value='mme'/>Mme
<input type='radio' name='civil' value='mlle'/>Mlle
```

  Dans un servlet, l'affichage de la liste des paramètres :

```
Enumeration<String> listParam = request.getParameterNames();
while(listParam.hasMoreElements()){
   out.println(listParam.nextElement());
}
```

Les informations sur le client

- Les informations sur le client concernent l'utilisateur et la machine ayant émis la requête. Elles sont toutes obtenues à partir de méthodes de l'objet `HttpServletRequest`

- Les informations sur la machine cliente sont :
  - son adresse ip, récupérée avec la méthode

```
public String getRemoteAddr()
```

  - son nom (nom de domaine complètement qualifié ou FQDN), récupéré avec la méthode

```
public String getRemoteHost()
```

Remarque : Si la requête est envoyée par un serveur proxy, ces méthodes retournent respectivement l'adresse ip ou le nom de celui ci.

Les informations sur le client

- Les informations sur l'utilisateur sont :
  - le type d'authentification mis en œuvre pour l'identifier, récupéré avec la méthode

```
public String getAuthType()
```

Cette méthode retourne :

- **BASIC_AUTH** authentification login/mot de passe avec la boite de dialogue du navigateur ;
- **DIGEST_AUTH** authentification basique avec mot de passe transformé par hachage ;
- **FORM_AUTH** authentification login/mot de passe avec un formulaire crée par le développeur ;
- **CLIENT_CERT_AUTH** authentification HTTPS (par certificat client).
- **null** si aucune authentification n'est utilisée.

Les informations sur le client

- le login utilisé lors de l'authentification :

```
public String getRemoteUser()
```

Cette méthode retourne **`null`** si l'utilisateur n'est pas authentifié.

- tester si le client possède le rôle spécifié :

```
public boolean isUserInRole(String role)
```

- le nom du client authentifié :

```
public Principal getUserPrincipal()
```

Cette méthode retourne **`null`** si l'utilisateur n'est pas authentifié.

Actions sur le client

- L'application web peut exécuter des actions sur l'utilisateur qui a émis la requête :
  - Demander à l'utilisateur se s'authentifier

    ```
    boolean authenticate(HttpServletResponse response)
    ```

    Cette méthode retourne **`false`** si l'utilisateur n'est pas enregistrer.

  - Authentifier l'utilisateur en se basant sur le username et le mot de passe qu'il a fourni :

    ```
    void login(String username, String password)
    ```

    Cette méthode lance une exception **`ServletException`** lorsque **`username`** ou **`password`** est invalide.

  - Effacer les informations concernant l'utilisateur authentifié

    ```
    void logout()
    ```

Les en-têtes HTTP de requête

Structure de la requête :

- requête GET

```
GET /nom_du_servlet?param1=valeur1&param2=valeur2&... HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/...
Accept: text/html,...
Referer: ...
  ...
```

- requête POST

```
POST /nom_du_servlet HTTP/1.1
Host: localhost:8080\r\n
User-Agent: Mozilla/...
Accept: text/html,...
  ...
param1=valeur1&param2=valeur2&...
```

Les en-têtes HTTP de requête

- Les en-têtes HTTP de requête sont des informations jointes à la requête par le navigateur.
- Elles sont parfois utiles pour le traitement de la requête.
- Les en-têtes de requête sont définies par le protocole HTTP. Voir RFC 2616

Les en-têtes HTTP de requête

- Les en-têtes HTTP les plus utilisées :
  - **`Accept`**,indique les formats de données que peut gérer le navigateur (**`"text/html"`**, **`"text/plain"`**, **`"image/gif"`**, **`"image/jpeg"`**, ...) ;
  - **`Accept-Language`**, indique par ordre de préférence les langues acceptées par client ;
  - **`Accept-Charset`**, indique le jeu de caractères acceptée par le client ;
  - **`Accept-Encoding`**, indique le type d'encodage de contenu supporté par le navigateur ;
  - **`Host`**, indique le nom du serveur ou son adresse ip, suivi du port comme spécifiés dans l'url ;

Les en-têtes HTTP de requête

- Les en-têtes HTTP les plus utilisées (suite) :
  - **`User-Agent`**, indique le navigateur utilisé par le client ;
  - **`Referer`**, indique l'url de la page ayant permis d'avoir la page courante ;
  - **`Cookie`**, spécifie une information (un cookie) à communiquer au serveur.

Méthodes pour la lecture des en-têtes de requête

Les en-têtes HTTP sont lues à l'aide de méthodes de `javax.servlet.http.HttpServletRequest` :

- `public String getHeader(String name)`, retourne la valeur :
  - de l'en-tête nommée,
  - `null` si l'en-tête ne figure pas dans la requête.
- `public long getDateHeader(String name)`, retourne la valeur :
  - de l'en-tête nommée,
  - `-1` si l'en-tête ne figure pas dans la requête.

  Exemple l'en-tête `If-Modified-Since` indique la date de dernière modification de la réponse.

Méthodes pour la lecture des en-têtes de requête

- `public int getIntHeader(String name)`, retourne la valeur
  - de l'en-tête nommée,
  - `-1` si l'en-tête ne figure pas dans la requête.

  Exemple : `content-length` indique en nombre d'octets la taille du fichier (ou des fichiers) présent(s) dans la requête.

- `public Enumeration<String> getHeaders(String name)` retourne l'ensemble des valeurs de l'en-têtes multi-valuée spécifiée.

- `public Enumeration<String> getHeaderNames()`, retourne l'ensemble des en-têtes contenues dans la requête.

Les en-têtes de requête

Exemple

Lire toutes les en-têtes HTTP contenues dans la requête :

```
Enumeration<String> enTetes = request.getHeaderNames();
while (enTetes.hasMoreElements()) {
    String nomEntete = enTetes.nextElement();
    String valeurEntete = request.getHeader(nomEntete);
    out.println("nomEntete + ": " + valeurEntete + "<br/>");
}
```

Les informations sur serveur

Un servlet peut avoir des informations sur le serveur qui l'exécute. Parmis ces informations il y'a :

- le nom de la machine serveur :

```
// Méthode de HttpServletRequest
public String getServerName()
```

- le port d'écoute :

```
// Méthode de HttpServletRequest
public int getServerPort()
```

- le nom et la version du logiciel serveur :

```
// Méthode de ServletContext
public String getServerInfo()
```

Informations sur la machine serveur

Exemple

Dans un servlet :

```
out.println("Nom du serveur: " + request.getServerName()+"<br/>");
out.println("Port d'écoute: " + request.getServerPort()+ "<br/>");
out.println("Nom et version du logiciel serveur: " +
    getServletContext().getServerInfo()+ "<br/>");
```

Affichage dans le navigateur :

Nom du serveur: localhost
Port d'écoute: 8080
Nom et version du logiciel serveur: Apache Tomcat/9.0.13

Autres informations

- IL y a des informations que le servlet peut attacher à la requête ou au contexte de l'application :
  - les attributs de requête,
  - les attributs de contexte.

- C'est un mécanisme qui permet au servlet de partager des informations (objets) avec d'autres servlets.

Attributs de requête

- Attacher un objet à la requête :

```
// Méthode de HttpServletRequest
void setAttribute(String nom, Object objet)
```

- Récupérer un objet (attribut) attaché à la requête :

```
// Méthode de HttpServletRequest
Object getAttribute(String nom)
```

Remarque

Cette méthode retourne **`null`** lorsque l'attribut n'existe pas dans la requête.

Attributs de requête

- Supprimer un attribut précédemment attaché à la requête :

```
// Méthode de HttpServletRequest
void removeAttribute(String nom)
```

- Obtenir la liste des attributs attachés à la requête :

```
// Méthode de HttpServletRequest
Enumeration<String> getAttributeNames()
```

Attributs de requête : transfert de requête

- Un servlet peut transférer une requête à un autre servlet.
- Les attributs attachés par le premier servlet seront alors disponibles pour le deuxième servlet.
- Le transfert de requête se fait à l'aide d'un objet **`javax.servlet.RequestDispatcher`**.

Exemple

`servlet1` transfert une requête à **`servlet2`**

```
// Dans servlet1
RequestDispatcher rd = request.getRequestDispatcher("url_Servlet2");
rd.forward( request, response );
```

Attributs de requête : transfert de requête

Remarques :

- Les attributs attachés à la requête ne sont disponibles que durant le traitement cette dernière.
- L'url donnée à la méthode `getRequestDispatcher` est relative au contexte de l'application.
  Par exemple, l'url `/url_Servlet2` ciblera implicitement le servlet `/contexte/url_Servlet2`.

Attributs de contexte

- Attacher un objet au contexte de l'application :

```
// Méthode de ServletContext
void setAttribute(String nom, Object objet)
```

- Récupérer un objet (attribut) précédemment attaché au contexte de l'application :

```
// Méthode de ServletContext
Object getAttribute(String nom)
```

Remarque

Cette méthode retourne `null` lorsque l'attribut n'existe pas dans le contexte.

Attributs de contexte

- Obtenir la liste des attributs attachés au contexte de l'application

```
// Méthode de ServletContext
Enumeration<String> getAttributeNames()
```

- Supprimer un attribut précédemment attaché au contexte de l'application :

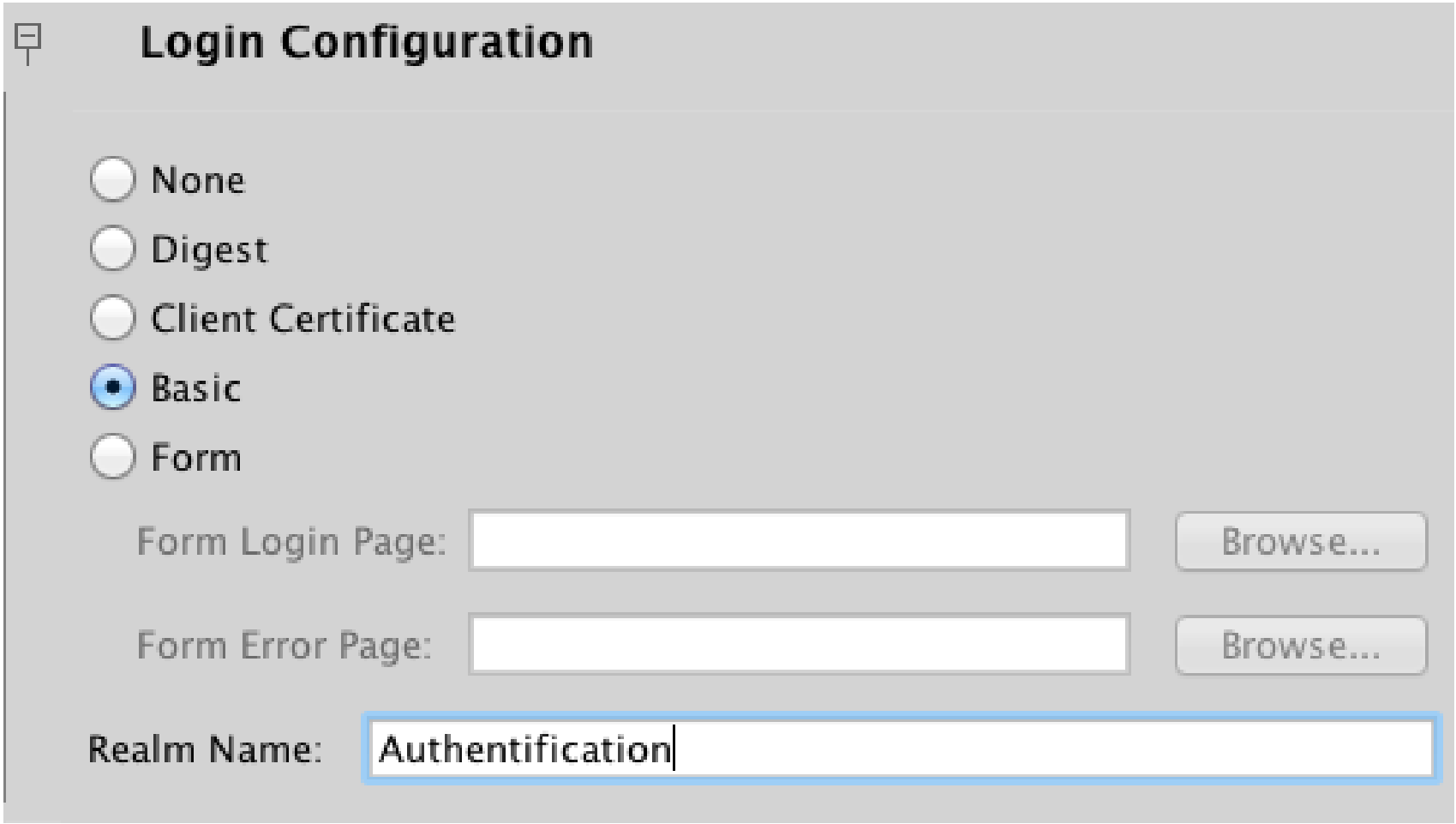
```
// Méthode de ServletContext
void removeAttribute(String nom)
```

Configurer une authentification `BASIC_AUTH`

La configuration d'une authentification type `BASIC_AUTH` sous le serveur tomcat se fait en 5 étapes :

1. Définir le type d'authentification ;
2. Définir les rôles ;
3. Spécifier les ressources à protéger (Créer une contrainte d'authentification)
4. Activer la contrainte d'authentification
5. Créer les utilisateurs

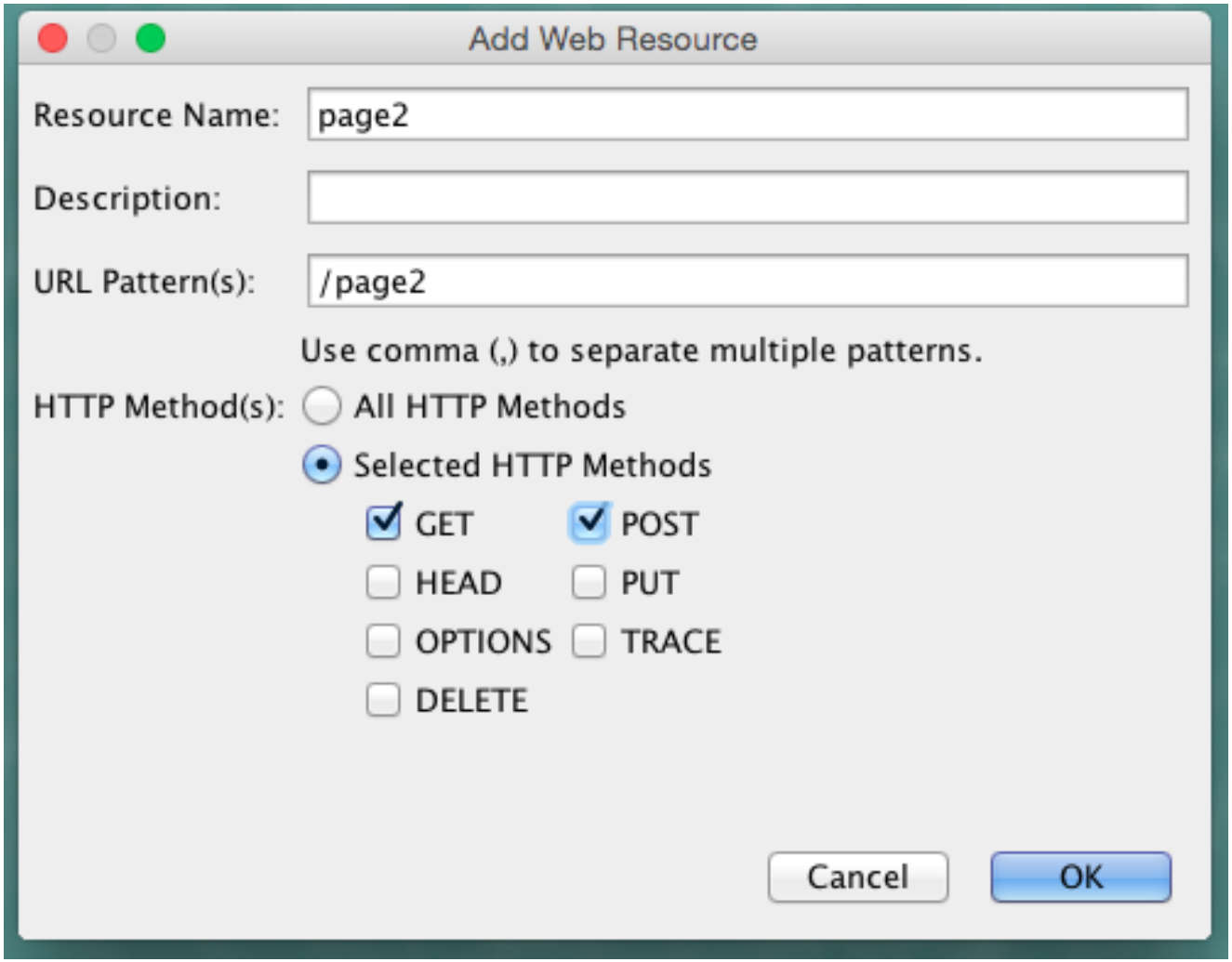
BASIC_AUTH : définir le type d'authentification



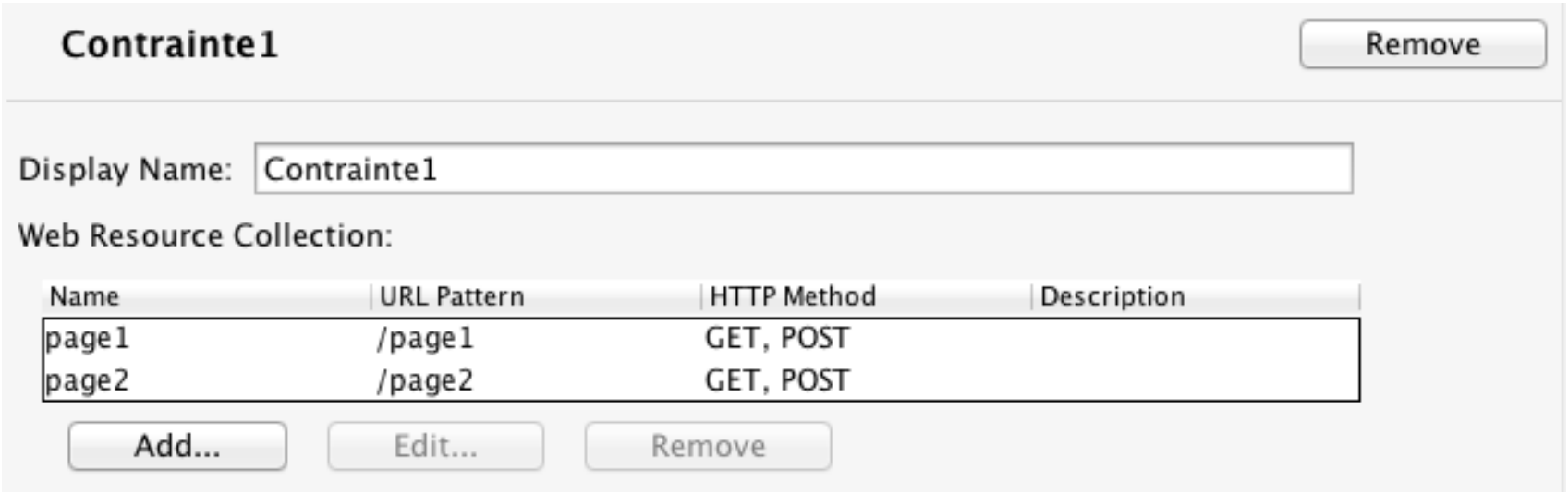
BASIC_AUTH : définir les rôles



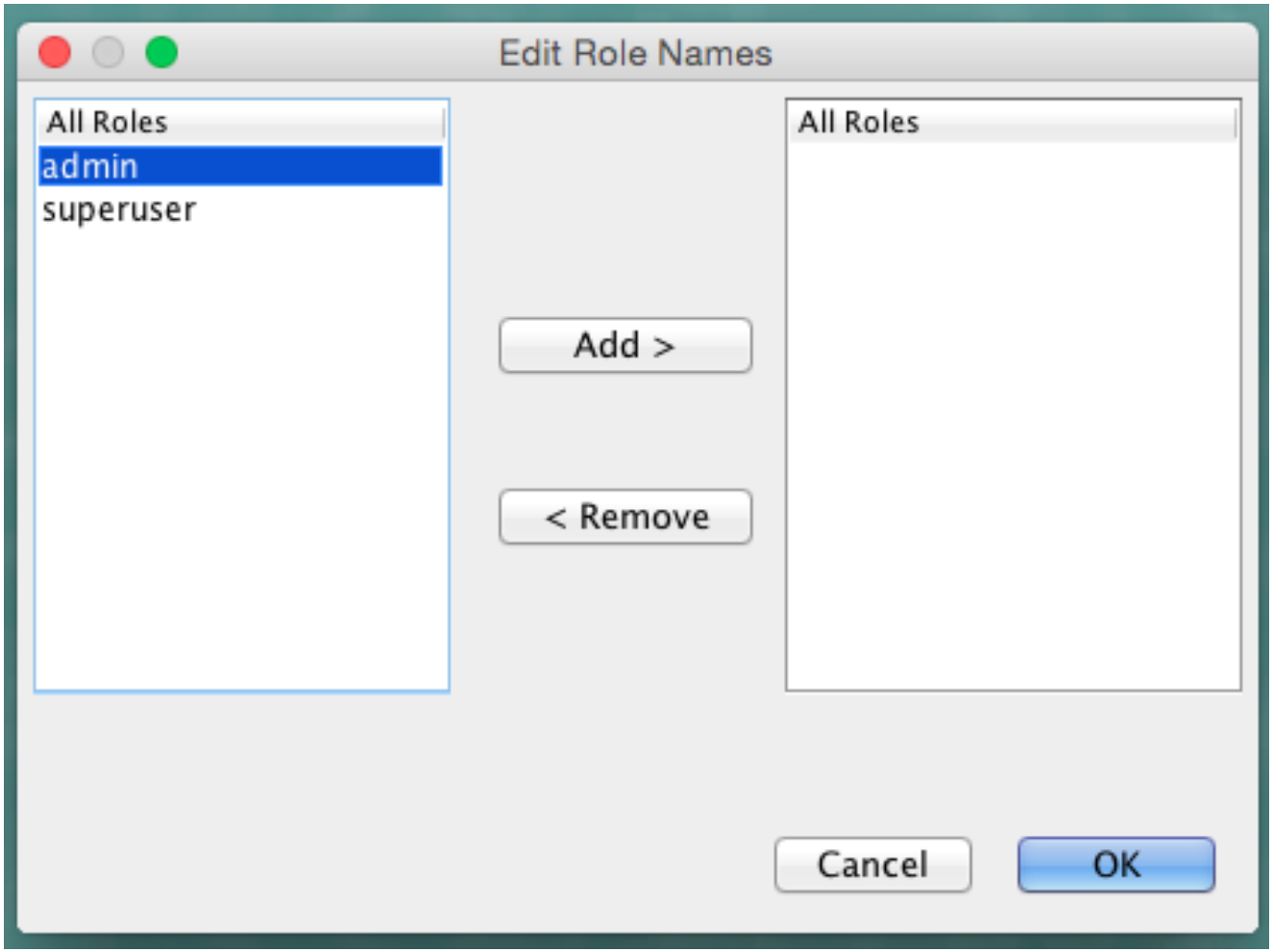
BASIC_AUTH : spécifier les ressources à protégées



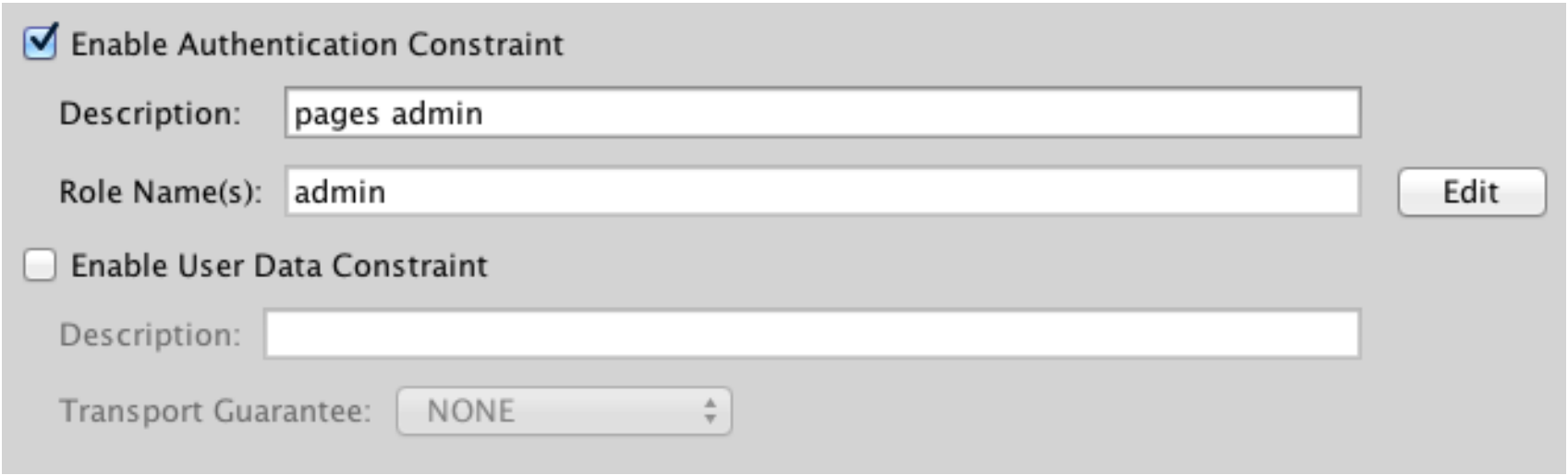
BASIC_AUTH : spécifier les ressources à protégées



BASIC_AUTH : Activer l'authentification



BASIC_AUTH : Activer l'authentification



BASIC_AUTH : Créer les utilisateurs

- Les utilisateurs ainsi que leurs rôles sont définis au niveau du serveur tomcat dans le fichier **tomcat-users.xml**
- Exemple :

```
<role rolename="superuser"/>
<role rolename="admin"/>
<user username="cosinus" password="1234" roles="superuser"/>
<user username="sinus" password="2345" roles="admin"/>
```

Configuration de `BASIC_AUTH` dans le fichier `web.xml`

```
<!- Le type d'authentification ->
<login-config>
   <auth-method>BASIC</auth-method>
   <realm-name>Authentification</realm-name>
</login-config>

<!- Les rôles ->
<security-role>
   <role-name>superuser</role-name>
</security-role>

<security-role>
   <role-name>admin</role-name>
</security-role>
```

Configuration de `BASIC_AUTH` dans le fichier `web.xml`

```
<!- Les autorisations de admin ->
<security-constraint>
    <display-name>Contrainte1</display-name>

    <web-resource-collection>
        <web-resource-name>page1</web-resource-name>
        <url-pattern>/page1</url-pattern>
    </web-resource-collection>

    <web-resource-collection>
        <web-resource-name>page2</web-resource-name>
        <url-pattern>/page2</url-pattern>
    </web-resource-collection>

    <auth-constraint>
        <role-name>admin</role-name>
    </auth-constraint>
</security-constraint>
```